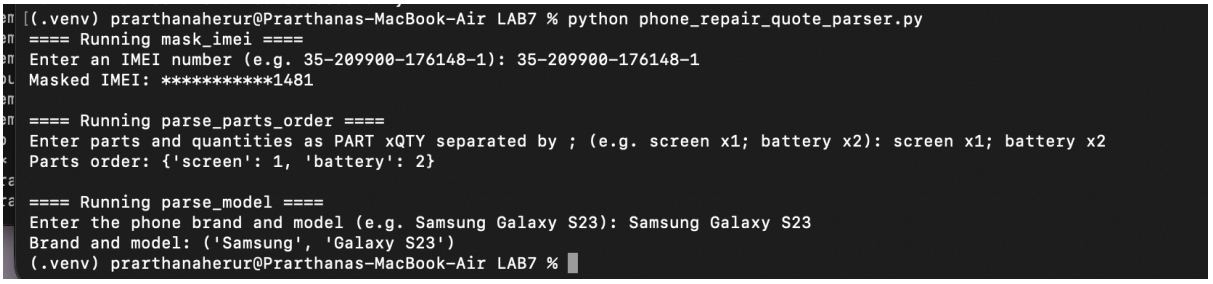
LAB 7: Fuzzing for Input Validation Bugs

NAME:PRARTHANA HERUR
SECTION:F
SRN:PES2UG24CS367

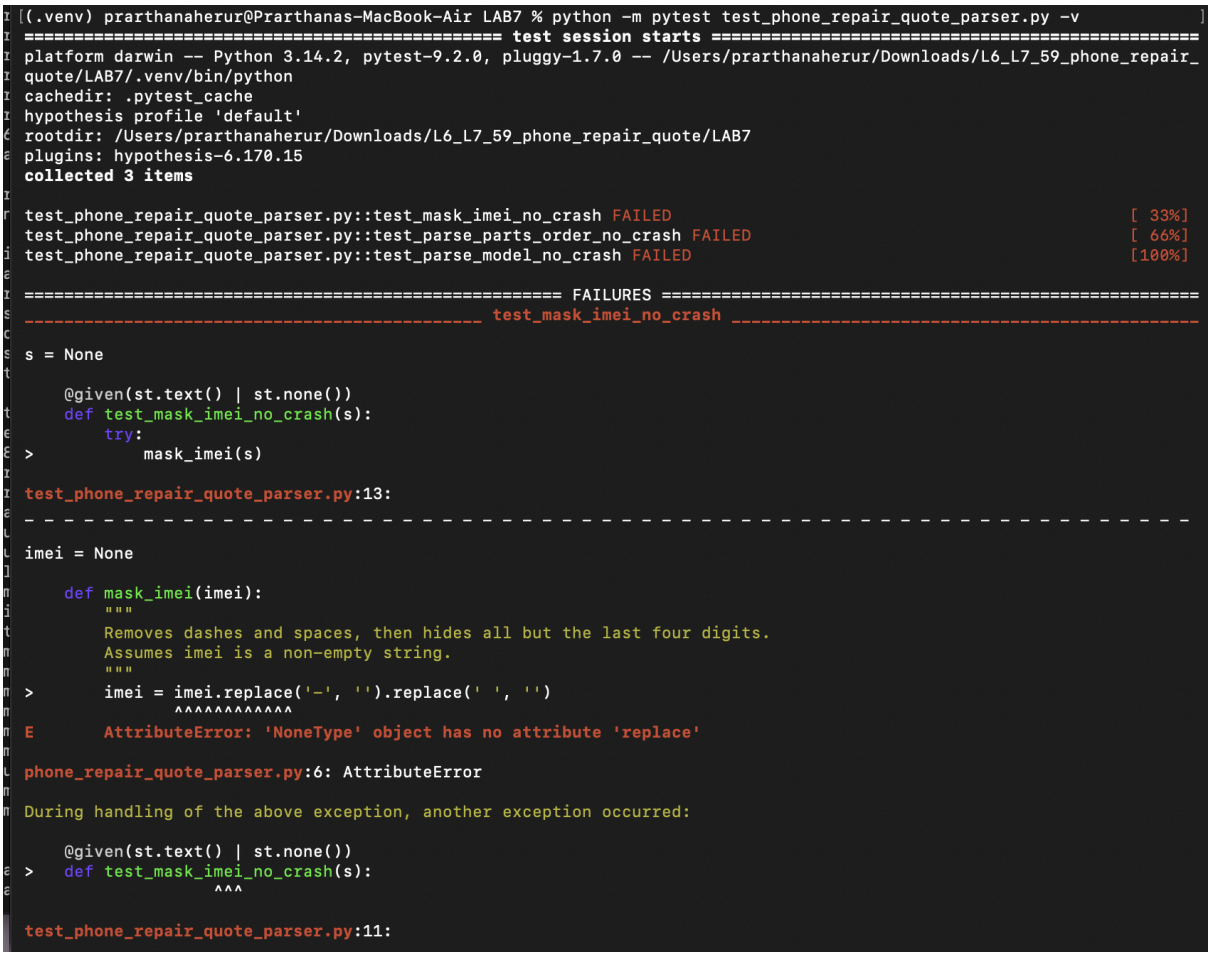
Original Application Execution



```
(.venv) prarthanaherur@Prarthanas-MacBook-Air LAB7 % python phone_repair_quote_parser.py
==== Running mask_imei ====
Enter an IMEI number (e.g. 35-209900-176148-1): 35-209900-176148-1
Masked IMEI: **********1481

==== Running parse_parts_order ====
Enter parts and quantities as PART xQTY separated by ; (e.g. screen x1; battery x2): screen x1; battery x2
Parts order: {'screen': 1, 'battery': 2}

==== Running parse_model ====
Enter the phone brand and model (e.g. Samsung Galaxy S23): Samsung Galaxy S23
Brand and model: ('Samsung', 'Galaxy S23')
(.venv) prarthanaherur@Prarthanas-MacBook-Air LAB7 %
```



```
(.venv) prarthanaherur@Prarthanas-MacBook-Air LAB7 % python -m pytest test_phone_repair_quote_parser.py -v
============================= test session starts ==============================
platform darwin -- Python 3.14.2, pytest-9.2.0, pluggy-1.7.0 -- /Users/prarthanaherur/Downloads/L6_L7_59_phone_repair_quote/LAB7/.venv/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/prarthanaherur/Downloads/L6_L7_59_phone_repair_quote/LAB7
plugins: hypothesis-6.170.15
collected 3 items

test_phone_repair_quote_parser.py::test_mask_imei_no_crash FAILED        [ 33%]
test_phone_repair_quote_parser.py::test_parse_parts_order_no_crash FAILED [ 66%]
test_phone_repair_quote_parser.py::test_parse_model_no_crash FAILED      [100%]

=================================== FAILURES ===================================
_____________________________ test_mask_imei_no_crash _____________________________

s = None

    @given(st.text() | st.none())
    def test_mask_imei_no_crash(s):
        try:
>           mask_imei(s)

test_phone_repair_quote_parser.py:13:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

imei = None

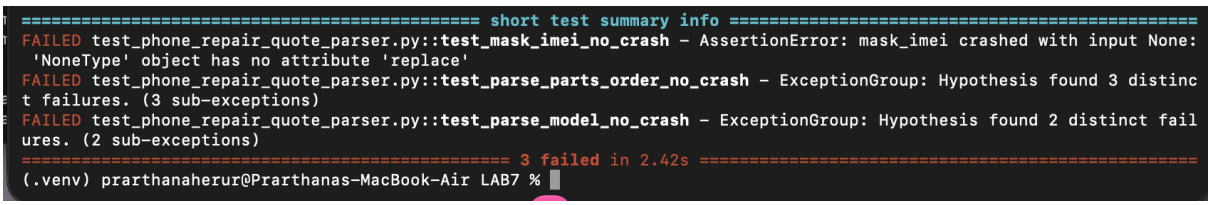
    def mask_imei(imei):
        """
        Removes dashes and spaces, then hides all but the last four digits.
        Assumes imei is a non-empty string.
        """
>       imei = imei.replace('-', '').replace(' ', '')
               ^^^^^^^^^^^^
E       AttributeError: 'NoneType' object has no attribute 'replace'

phone_repair_quote_parser.py:6: AttributeError

During handling of the above exception, another exception occurred:

    @given(st.text() | st.none())
>   def test_mask_imei_no_crash(s):
                ^^^

test_phone_repair_quote_parser.py:11:
```



Implementation of Hypothesis Tests

The existing file `test_phone_repair_quote_parser.py` contained three incomplete test functions. These functions were completed using the Hypothesis library.

The following tests were implemented:

- `test_mask_imei_no_crash()`
- `test_parse_parts_order_no_crash()`
- `test_parse_model_no_crash()`

The Hypothesis strategy `st.text() | st.none()` was used to generate random strings and None values.

Try-except blocks were implemented in each test to identify unexpected exceptions. When an exception occurred, the test was marked as failed and displayed the input responsible for the failure.

Bug Fixes and Code Modifications

The original application was modified to handle unexpected and invalid inputs safely.

The following changes were implemented:

mask_imei():

- Added type validation to handle non-string inputs.
- Added empty-input validation.
- Used `max()` to prevent negative string repetition when masking short inputs.
- Preserved the original functionality of removing spaces and hyphens and masking the IMEI.

parse_parts_order():

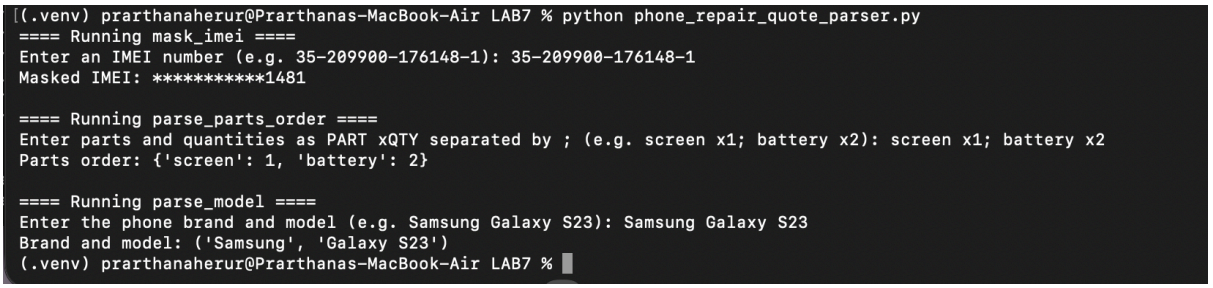
- Added validation for None and empty inputs.
- Added checks for missing separators and empty entries.
- Used `rsplit()` to separate the part name and quantity.

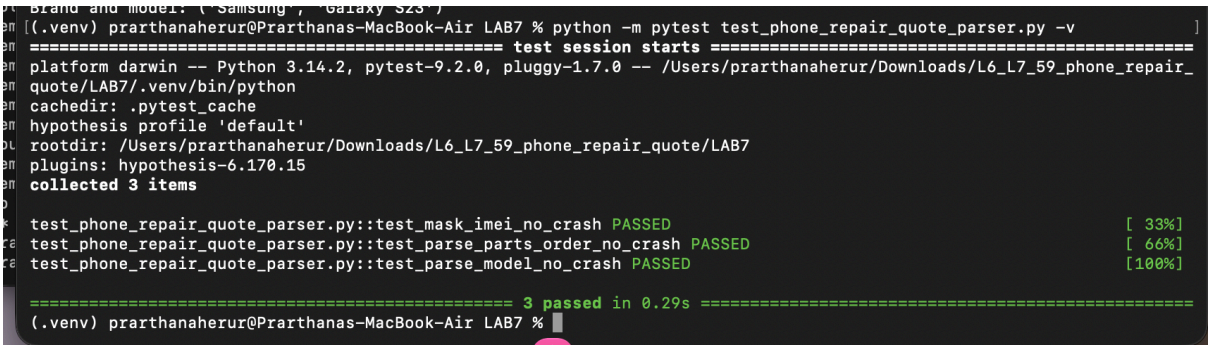
- Added exception handling for invalid integer conversions.
- Added validation to ignore zero and negative quantities.
- Invalid entries are skipped instead of causing the application to crash.

parse_model():

- Added type validation.
- Added checks for empty input and inputs containing fewer than two words.
- Used `split()` to handle multiple spaces.
- Preserved the original brand and model extraction functionality.

Execution of Fixed Application





Conclusion

The Phone Repair Quote Parser application was successfully tested using Hypothesis-based fuzz testing. Three functions were tested with automatically generated strings and None values.

The initial testing identified input validation bugs in all three functions. These bugs were corrected using type checking, conditional statements, input validation and exception handling.

After implementing the corrections, the application executed successfully, and all three Hypothesis tests passed.

The experiment demonstrated the practical use of automated fuzz testing to identify unexpected input-related failures and improve the robustness of Python applications.